فرزاد دهقان منشادی 40252125

دستورcheckout

دستور git checkout بر روی سه موجودیت متمایز عمل می کند: فایل ها، commit ها و شاخه ها. علاوه بر تعریف"checkout" ، معمولا عبارت "checking out" برای استفاده از دستور git checkout نیز به کار برده می شود.

Checking outشاخه ها

دستور git checkout به شما امکان می دهد بین شاخه های ایجاد شده توسط git branch حرکت کنید .Checking out یک شاخه، فایل ها را در working directory به روز می کند تا با نسخه ذخیره شده در آن شاخه مطابقت داشته باشد و به Git می گوید که تمام commit های جدید را در آن شاخه ثبت کند. فکر کنید که این راهی است برای انتخاب خطی از توسعه که شما روی آن کار می کنید .

کاربرد: شاخه های موجود

با فرض اینکه مخزنی که در آن کار می کنید شامل شاخه های قبلی باشد، می توانید با استفاده از git checkout بین این شاخه ها جا به جا شوید. برای این که بفهمید چه شاخه هایی موجود است و نام شاخه فعلی چیست، git branch را اجرا کنید.

شاخه های جدید

Git checkoutبه صورت دستی با git branchکار می کند. از دستور git branch می توان برای ایجاد شاخه ی جدید استفاده کرد. وقتی می خواهید ویژگی جدیدی را شروع کنید، با استفاده از *git branch new_branch* شاخه ی جدیدی را ایجاد می کنید. پس از ایجاد می توانید از *git checkout new_branch* برای رفتن به آن شاخه استفاده کنید. همچنین، دستور git checkout پارامتر -b را می پذیرد که به عنوان روشی راحت عمل می کند که شاخه ی جدید را ایجاد و سریع به آن سوئیچ می کند. با جا به جایی بین آن ها با git checkout می توانید روی چندین ویژگی در یک مخزن واحد، کار کنید.

git checkout -b <new-branch>

مثال فوق هم زمان، ایجاد و check out می کند. گزینه -b یک پرچم (flag) راحتی است که به Git می گوید قبل از اجرای git branch، git checkoutرا اجرا کند.

git checkout -b <new-branch> <existing-branch>

به طور پیش فرض *git checkout -b* شاخه ی جدید را در HEAD فعلی قرار می دهد. یک پارامتر شاخه ی اضافی اختیاری می تواند به git checkout منتقل شود. در مثال فوق، تصویب شده است که existing-branch ، new-branch (شاخه موجود) را به جای HEAD فعلی پایه گذاری می کند .

Git Checkoutیک شاخه ی از راه دور

هنگام همکاری با یک تیم، استفاده از مخزن های از راه دور؛ عادی است. این مخزن ها ممکن است میزبانی و به اشتراک گذاشته شوند یا کپی محلی همکار دیگر باشند. هر مخزن از راه دور، مجموعه شاخه های خاص خود را خواهد داشت. برای checkout شاخه ی از راه دور، ابتدا باید محتوای شاخه را واکشی (fetch) کنید .

git fetch --all

می توانید یک شاخه محلی جدید را checkout کنید و آن را به آخرین commit شاخه ی از راه دور، تنظیم مجدد (reset) کنید.

git checkout -b <branchname>

git reset --hard origin/<branchname>

-f(--force)

برای زمان هایی که هنگام تعویض شاخه فایل های ردیابی نشده در مسیر وجود داشته باشد

--guess

اگربرنچی با این اسم توی برنچ های خودمون پیدا نشد و با همین اسم در remot وجود داشت آن را بیاور

Git stach

Git stash، تغییرهایی را که در نسخه ی کاریتان ایجاد کرده اید، به طور موقت ذخیره می کند تا بتوانید روی ویژگی دیگری کار کنید. سپس در آینده هر موقع نیاز شد، برگردید و آن ها را دوباره اعمال کنید .Stash کردن (ذخیره کردن) زمانی سودمند است که می خواهید روی زمینه ی دیگری کار کنید اما در وسط تغییر کدی هستید و هنوز آماده ی commit کردن نیستید و بعد آن میره روی آخرین تغییراتی که کامیت کرده اید.

Stash کردن

دستور git stash تغییرهای commit نشده تان را (هم آن هایی که stage شده اند و هم آن هایی که stage نشده اند) می گیرد؛ برای استفاده ی بعدی، ذخیره شان می کند و سپس به نسخه ی کاریتان برمی گرداند.

- می توانید با دستور git stash pop تغییرهای stash شده خود را دوباره اعمال کنید.
- همچنین، می توانید با دستور git stash apply، تغییرهایی را هم اعمال کرده و هم در stash نگه دارید.
  این دستور هنگامی مفید است که بخواهید تغییرهای stash شده را روی چند شاخه اعمال کنید.
- اضافه کردن -u (یا --include-untracked) به دستور git stash، موجب stash شدن فایل های ردیابی نشده نیز می شود
- همچنین می توانید با اضافه کردن -a (یا --all) به دستور git stash تغییرهای فایل های نادیده گرفته شده را نیز stash کنید.

به طور پیش فرض، اجرای دستورgit stash ، موارد زیر را stash خواهد کرد:

- تغییرهایی که به فهرست شما اضافه شده اند (تغییرهای stage شده)
- تغییرهای ایجاد شده در فایل هایی که در حال حاضر git؛ آن ها را ردیابی می کند (تغییرهای stage نشده).

اما git موارد زیر را stash نمی کند:

- فایل هایی که نادیده گرفته شده اند.

مدیریت چندینstash

شما به اجرای یک stash محدود نیستید و می توانید دستور git stash را چندین بار اجرا کنید تا چندین stash داشته باشید. سپس از git stash listبرای مشاهده ی آن ها استفاده کنید.